

Reasoning Bank: State Recovery & Execution Ledger Protocol

1. Executive Purpose

The **Reasoning Bank** operates as the System Flywheel and RLVR (Reinforcement Learning via Verifiable Rewards) state tracking engine of #d.u.m.b.a.s.s. It guarantees that whenever an agent execution loop crashes, suffers an Android Out-Of-Memory (LMK) event, or gets paused by the operator, the exact execution path can be recovered without restarting from scratch or duplicating side-effects.

2. Core Recovery Principles

1. Deterministic Checkpointing:

- Every state transition must emit a checkpoint record to `active_execution_paths.json` and the local SQLite `active_task_state` table.
- A checkpoint contains: `step_index`, `action_name`, `input_state_hash`, `output_state_hash`, and `resume_token`.

2. Rollback & Unwind Protocol:

- If a step fails verification (e.g. script syntax error or broken file write), the agent unwinds to the last verified step hash (`rollback_target_step`).
- Any temporary filesystem mutations are rolled back using local git checkpoints or reverse diffs.

3. Verifiable Reward Grading:

- Completed trajectories are evaluated by the auditing loop against baseline assertions.
 - Trajectories earning a verified `+1.0` score are committed to the long-term training bank on Node Beta.
 - Trajectories earning `-1.0` are immediately moved to `quarantined/` with a full traceback.
-

3. Crash Recovery Runbook

When an agent reboots:

1. Query SQLite `SELECT * FROM active_task_state WHERE status = 'IN_PROGRESS' ORDER BY updated_at DESC LIMIT 1`.
2. Inspect `active_execution_paths.json` for the matching `session_id`.
3. Load the `resume_token` and resume execution directly from `current_step`.
4. Inform the user of recovery without re-executing completed milestones.